Design Parking Lot

Reference:
Video: Design Parking Lot with Complete Implementation (English)
Git link: src/main/java/com/conceptcoding/interviewquestions/parking_lot · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Lets understand the flow first:

- Vehicle arrives at an Entry gate .
  - We can have multiple entry gate

- Based on Vehicle Type, particular parking spot is looked for.
  - Interviewer can ask for flexibility in choosing the parking spot like random, or any specific parking strategy
  - Also interviewer can say, parking lot can have multiple levels.
  - Concurrency should be handled properly, for example:
    - Same parking spot can not be reserved by two different vehicles.
    - for 1 vehicle say two wheeler we should not block the whole parking lot itself

Concept && Coding

- If Parking spot is available, Ticket is generated for particular vehicle
  a. Ticket can have: vehicle no, parking level no, parking spot no and Entry time.

- At Exit gate, cost computation should have happened, Interviewer can ask for different strategies for cost computation like:
  a. Fixed price
  b. Hourly based computation etc.

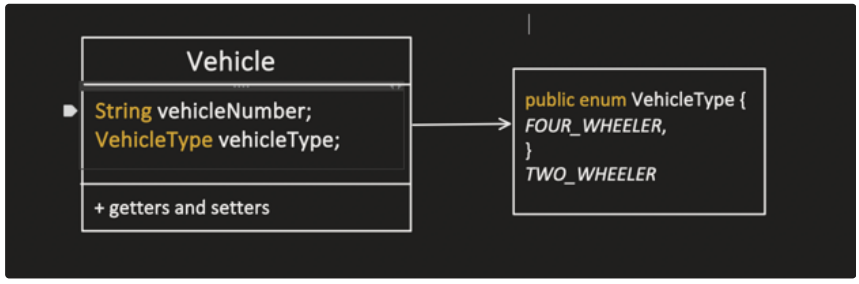
- Payment will be made against the ticket at Exit gate and Parking spot is free again.

Based on above understanding, we identified the below major objects involved in this design:

- Vehicle
- Parking spot
- Parking level
- Entry gate
- Exit Gate
- Ticket
- Payment

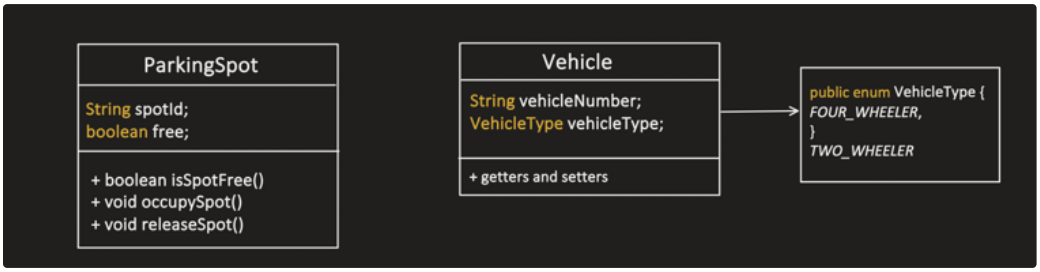
Lets start creating the UML:

1. **Vehicle**



Concept && Coding

2. **ParkingSpot**



ParkingSpot object is not intelligent, its just a POJO.

Also we can have multiple Parking spot (like : 2Wheeler, 4Wheeler, Truck, EV etc.)
We need some other intelligent object,

That's where **ParkingSpotManager** comes into the picture.

3.ParkingSpotManager

Its an intelligent object, which help in managing Parking Spots like add, remove, park, unPark, search free spot etc.

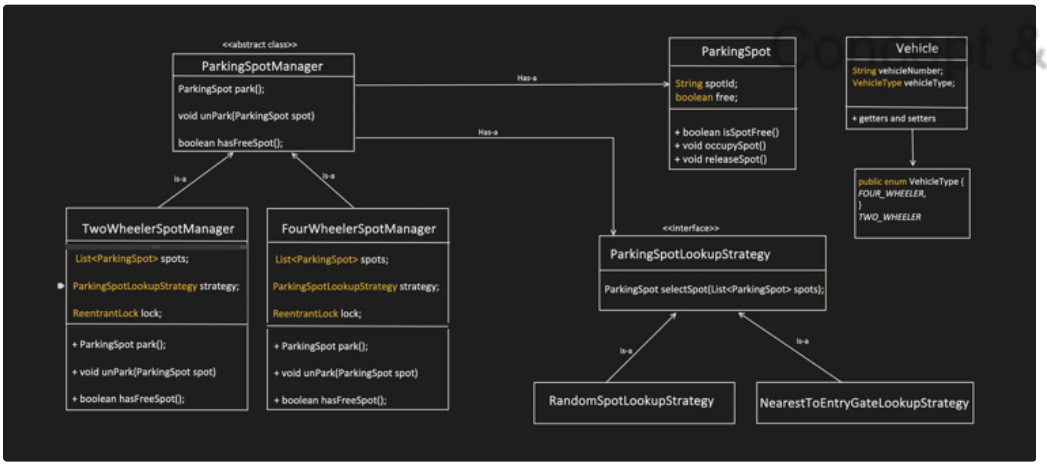
Intention:

We will have different manager to manage different type of parking.

Reasons:

- Each manager has dedicated task to manage their parking spots only.
  - They will search for free space only in their set itself.
  - If they need to put lock during park() and unPark() operation, they can only block their set of spots only.

- Each manager can use different Parking Spot look-up strategy.
  - For ex: 2Wheeler manager is using Random strategy, where as 4Wheeler manager is using NearestToEntraceGate Strategy.



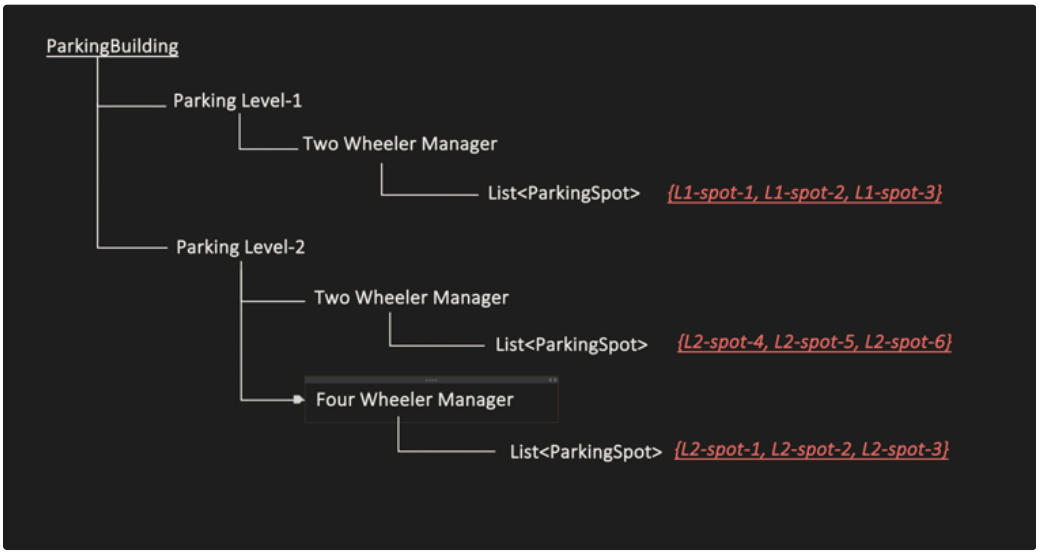
4.ParkingLevel

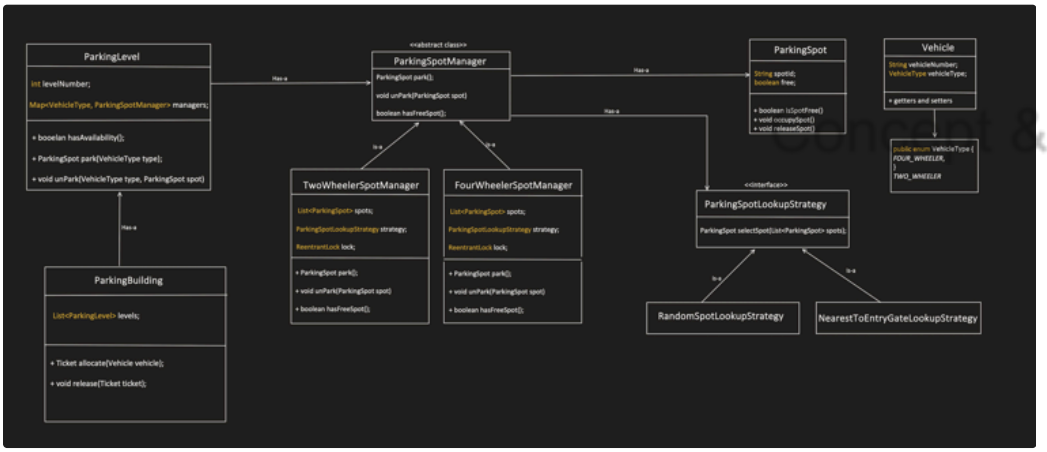
In case of Multi-level parking each level:

- manages their parking independently.

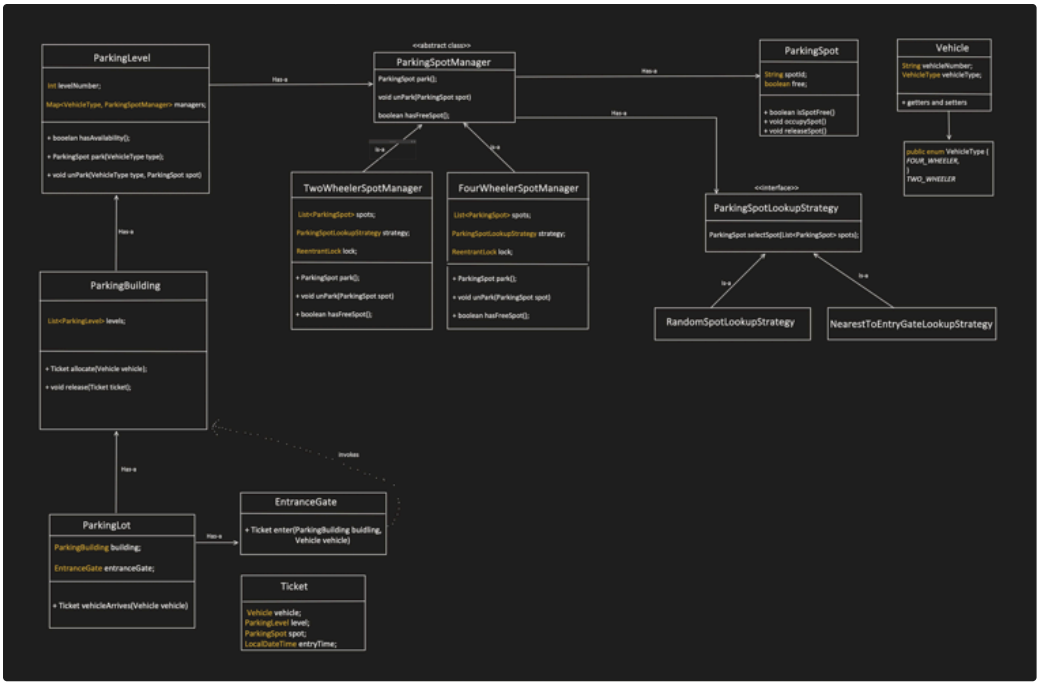
- Each level could support different type of parking, for ex: Level-1 support only 2wheelers, Level-2 supports both 2wheeler and 4Wheelers likewise.

- Also we don't have to lock the whole building parking spots itself, only specific Level and specific Manager (2Wheeler, 4Wheeler etc.) need to be locked during parking and unParking operation.





5.ParkingLot and EntranceGate

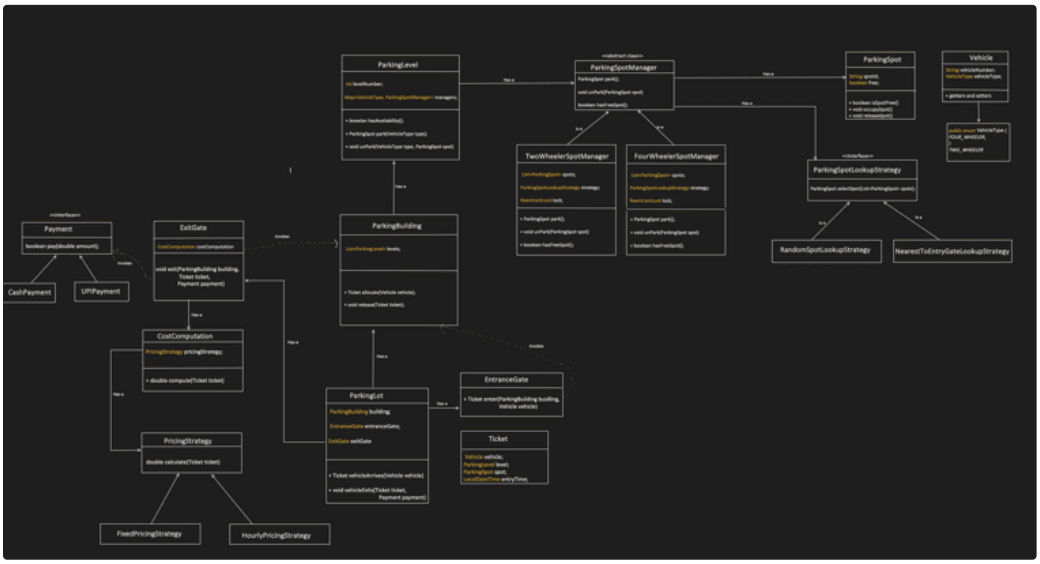


6.Exit Gate

3 things happened at Exit gate:

Concept && Coding

- Based on ticket, Cost computation
- Payment
- On payment success, release of the reserved spot



Reference:

Video: Design Parking Lot with Complete Implementation (English)

Git link: src/main/java/com/conceptcoding/interviewquestions/parking_lot · main · shrayansh jain / LLD-LowLevelDesign · GitLab

Concept && Coding